



1

5

7.5 sp

TD

كلية الهندسة المعلوماتية

السنة الأولى

أساسيات لغة Java

د. ماهر صارم



RB Informatcs; 30/3/2026

محتوى مجاني غير مخصص للبيع التجاري

البرمجة 2

لماذا سندرس الـ OOP بلغة جافا وليس بلغة ++c؟

لأن لغة جافا أكثر تقدماً وتطوراً من لغة ++c، ولغة جافا هي لغة كائنية التوجه بشكل كامل على عكس ++c، وهذا سيسهل علينا التعامل مع OOP والتركيز على المفاهيم والمنطق البرمجي أكثر.



إضافة آريسيزية:

ما هو الـ OOP (البرمجة كائنية التوجه)؟

هو أسلوب برمجي يعتمد على تقسيم البرنامج إلى وحدات مستقلة تسمى كائنات (objects) حيث يجمع كل كائن بين البيانات والأفعال (methods) الخاصة به لتسهيل تنظيم الكود.

الفرق بين لغة ++c ولغة Java:

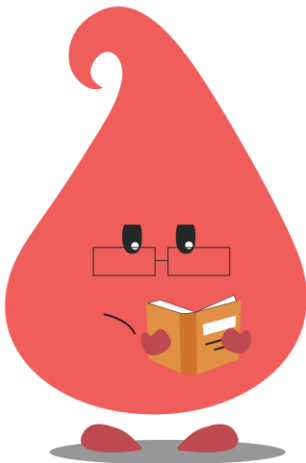
الإدخال والإخراج:

Java	++c	
أولاً يجب استدعاء class معين وهو Scanner وبعدها صنع object في هذا الصف، وهناك عدة طرق للإدخال حسب نوع المتغير.	cin>>	الإدخال
عن طريق تعليمة System.out.print() التي تقوم بطباعة ما بين القوسين على نفس السطر	cout<<	الإخراج
أو هذه التعليمة System.out.println() التي تقوم بطباعة ما بين القوسين ثم النزول سطر للأسفل.		

مثال توضيحي:

```
1 import java.util.Scanner;
2 public class Main {
3     public static void main(String[] args) {
4         java.util.Scanner input=new Scanner(System.in);
5         int a;
6         double b;
7         a=input.nextInt();
8         b=input.nextDouble();
9     }
10 }
```

شرح الكود:



- 1: استدعاء صف scanner المسؤول عن الإدخال.
- 2+3: سطور أساسية في لغة Java.
- 4: تعريف كائن (object) من الصف (class) Scanner باسم input
- 5+6: تعريف متغيرين a من نوع int و b من نوع double.
- 7: عملية الإدخال ونلاحظ أننا كتبنا اسم الكائن وبعده
nextInt وهو int مثل نوع المتغير a.
- 8: نفس العملية في السطر السابع لكن من أجل متغير من نوع double.

إضافة:

- لإدخال string نستعمل nextLine أو next().
- لإدخال char نستعمل نفس التعليمات السابقة.
- لإدخال int نستعمل nextInt().
- لإدخال double نستعمل nextDouble().
- لإدخال Boolean نستعمل nextBoolean().

❖ تعريف المتغيرات:

نفسها باللغتين لكن هناك فرق فقط بتعريف:

- string في جافا يجب كتابة S حرف كبير أما في c++ لا.
- في c++ المتغير bool أما في جافا يكتب boolean.

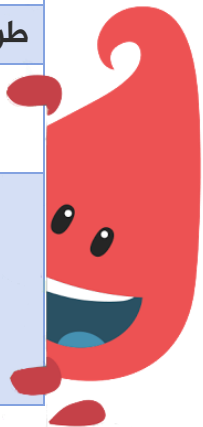


❖ البنى الشرطية والمصفوفات:

البنى الشرطية والحلقات نفسها لا تتغير بين اللغتين.
المصفوفات تختلف فقط بطريقة تعريف المصفوفة.

مقارنة عامة بين لغة c++ وJava:

Java	c++	
عبر تعليمة System.out.print() أو System.out.println()	عبر تعليمة cout<<	الإخراج
استدعاء صف scanner وإنشاء كائن (object) منه	عبر تعليمة cin>>	الإدخال
تكتب String (S حرف كبير)	تكتب string (s حرف صغير)	طريقة تعريف متغير string
تكتب boolean	تكتب bool	طريقة تعريف متغير Boolean
نقوم بإسناد حسب النوع (مثلاً للمتغير int نكتب nextInt() ويتم الإسناد للمتغير المراد إدخال القيمة له	مباشرة بعد cin>>	طريقة الإدخال



تمرين: مقارنة بين c++ و Java عند كتابة برنامج جمع عددين:

لغة Java

```

1 import java.util.Scanner;
2 public class Main {
3     public static void main(String[]
args) {
4         int a,b,c;
5         Scanner input=new Scanner
(System.in);
6         a=input.nextInt();
7         b=input.nextInt();
8         c=a+b;
9         System.out.print("sum="+c);
    }
}

```

لغة c++

```

1 #include <iostream>
2 using namespace std;
3 int main()
4 {
5     int a,b,c;
6     cin>>a;
7     cin>>b;
8     c=a+b;
9     cout<<c;
10    return 0;
11 }

```

شرح برنامج java:

1: استدعاء الصف المسؤول عن الإدخال.

2+3: سطور أساسية في لغة java.

4: تعريف متغيرات a,b,c من نوع int.

5: تعريف كائن من الصف scanner باسم input. (ملاحظة: يمكن تغيير اسم الكائن)

6+7: عملية إدخال المتغيرين a و b من النوع int.

8: عملية جمع a و b وإسنادها لـ c.

9: إشارة + تستخدم لربط النصوص مع المتغيرات في تعليمة طباعة واحدة بدون استخدام + المترجم (compiler) لن يستطيع فهم العلاقة بينها وسيعطي خطأ برمجي.

ملاحظة:

- في السطر 3 يمكننا تغيير الاسم بعد string[] ولن نجد خطأ compiler، أي لو غيرناها إلى a string[] سيستمر البرنامج بالعمل لأن ما بعد string يغير اسم المصفوفة.
- في السطر الخامس new scanner تدل على أننا قد صنعنا كائن جديد من الصف scanner و (System.in) تعني أنه يأخذ الدخل من Buffer والـ Buffer: هو المكان الذي يخزن فيه المعلومات القادمة من لوحة المفاتيح مؤقتاً.

تمرين: برنامج لطباعة عدد int بالعكس (أي تحويل 1234 إلى 4321):

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        int remaining,result=0,num;
        Scanner input = new Scanner(System.in);
        num=input.nextInt();
        while(num > 0)
        {remaining=num%10;
          num=num/10;
          result=result*10+remaining;
        }
        System.out.println(result);
    }
}
```

شرح البرنامج:

- 7: حلقة while تتحقق إذا كان الرقم يساوي الصفر أو لا.
- 8: نقوم بأخذ باقي قسمة الرقم على 10 وهذا الباقي سيكون مساوي للآحاد.
- 9: نقسم الرقم على 10 وبما أن الرقم من نوع int فالناتج سيكون من دون ما بعد الفاصلة أي يأخذ الجزء الصحيح ويتجاهل كل ما يوجد يمين الفاصلة.
- 10: سيكون الناتج القديم مضروب بـ 10 حتى يزيد خانة وتصبح الآحاد تساوي الصفر، عندها نضيف الباقي للناتج ليحل محل الآحاد.
- (تكرر هذه الحلقة حتى يصبح العدد كله بعد الفاصلة عندها سيعتبر مساوياً للصفر)
- 12: عندما يصبح العدد مساوياً للصفر تنتهي الحلقة ويتم طباعة الناتج.

ملاحظة:

قمنا بوضع قيمة 0 للمتغير result لأنه لا يمكننا استعمال متغير لا قيمة له في java.

انتهت المحاضرة الأولى... بالتوفيق ☺

